

02_run_2D

August 31, 2026

1 02 – Run a 2D thermal simulation

Equivalent to run_2D_clean.ipynb, but using the generic PIC module. Supports NoMaxwell, SpectralMaxwell, and SBPMaxwell.

```
[ ]: using Pkg
      Pkg.activate(joinpath(@__DIR__, ".."))
      Pkg.instantiate()
```

```
[ ]: cd(@__DIR__)
      include(joinpath("../src", "PIC.jl"))
      using .PIC
      using Plots
      using Statistics
      using Base.Threads
      using Printf
      using FileIO, JLD2
      using SummationByPartsOperators
      println("nthreads = ", nthreads())
```

1.1 Setup grid and load initial data

```
[ ]: # Run parameters
      const N_exp = 5
      const N = 10^N_exp
      const order = 5
      const factor = 2

      J = (50, 50)
      box = Box((0.0, 1.0, 0.0, 1.0))
      g = PICGrid(box, J, order)

      nm = [1, 1]
      alpha_exp = 1
      _exp = 3
      = 10.0^(-_exp)
```

```

data_name = ␣
    ↪ "par_dis_rel_thermal_nm_[$(nm[1]),$(nm[2])]_alp$(alpha_exp)_N$(N_exp)_Th$( _exp) "
file_name = joinpath("../", "Initial_Distributions", data_name * ".jld2")

par_dis, run_name, pars, par_f_x, box_x, par_f_p, box_p = ␣
    ↪ retrieve_initial_data_D(file_name)

# Evolution parameters
t_i = 0.0
t_f = 1.0
M = 2001
M_g = 100
dt = (t_f - t_i) / (M - 1)
dx = differentials(g)
@show dt, minimum(dx), dt / minimum(dx)

```

1.2 Build state vector, deposit density, solve Poisson

```

[ ]: u = make_state(g, N)
u[1:4N] .= par_dis

n = zeros(size(g))
dep = DepositDensity(g, N)
deposit!(dep, u, g, n; factor=factor)
rho = n .- 1.0

E = zeros(2, size(g)...)
solve_poisson!(E, rho, g)
B = zeros(size(g))
set_fields!(u, g, N, E, B)
coordinate_test(u, g, N)

err_max_initial, err_l2_initial = check_constraints(u, g, N, dep, factor)
println("initial max |div E - rho| = ", err_max_initial)
println("initial L2 |div E - rho| = ", err_l2_initial)

surface(E[1,:,:], title="initial E1")

```

1.3 Choose Maxwell solver and prepare evolution

```

[ ]: # Choose one of the three Maxwell solvers:
#     1. NoMaxwell{2}()           -> electrostatic (B not evolved)
#     2. SpectralMaxwell(g)       -> periodic spectral derivatives
#     3. SBPMaxwell((Dx, Dy))    -> summation-by-parts finite differences

derivative_order = 4
solver_choice = :sbp # <-- change to :no_maxwell, :spectral, or :sbp

```

```

if solver_choice == :no_maxwell
    maxwell = NoMaxwell{2}()
    suffix = "no_maxwell"
elseif solver_choice == :spectral
    maxwell = SpectralMaxwell(g)
    suffix = "spectral"
elseif solver_choice == :sbp
    Dx = periodic_derivative_operator(derivative_order=derivative_order,
    ↪accuracy_order=6,
                                xmin=box.lo[1], xmax=box.hi[1], N=J[1])
    Dy = periodic_derivative_operator(derivative_order=derivative_order,
    ↪accuracy_order=6,
                                xmin=box.lo[2], xmax=box.hi[2], N=J[2])
    maxwell = SBPMaxwell((Dx, Dy))
    suffix = "sbp"
else
    error("unknown solver_choice = $solver_choice")
end

p = RHSPParams(g, N; factor=factor, maxwell=maxwell)
ws = StepWorkspace(u)

run_name_full =
    ↪"thermal_rel_${suffix}_J$(J[1])x$(J[2])_N$(N_exp)_Th$( _exp)_alp$(alpha_exp)_o$(order)"
out_file = joinpath("../", "Results", run_name_full * ".jld2")
mkpath(dirname(out_file))
isfile(out_file) && rm(out_file)

println("Running with solver: ", solver_choice)
println("Output: ", out_file)

err_max, err_l2 = check_constraints(u,g,N,dep, factor)

```

1.4 Time integration with RK4

```

[ ]: # save metadata
save(out_file, Dict(
    "run_name" => run_name_full,
    "par_grid" => (N, J, box, order),
    "par_evolv" => (t_i, t_f, M, M_g, dt)
))

# initial averages
save_averages(out_file, 1, u, g, N; dep=dep, factor=factor)

let t = t_i

```

```

out_every = div(M - 1, M_g - 1)
for k in 2:M
    RK4_step!(rhs!, u, t, dt, p; ws=ws)
    t += dt
    if (k - 1) % out_every == 0
        j = (k - 1) ÷ out_every + 1
        save_averages(out_file, j, u, g, N; dep=dep, factor=factor)
        EK, EE = total_energy(u, g, N)
        println("j = ", j, " t = ", round(t, digits=3), " E_total = ", EK,
↪+ EE)
    end
end
end

err_max_final, err_l2_final = check_constraints(u, g, N, dep, factor)
println("final    max |div E - rho| = ", err_max_final)
println("final    L2 |div E - rho| = ", err_l2_final)
println("Done. Output: ", out_file)

```

1.5 Final density snapshot

```

[ ]: # final density snapshot at the last saved time
deposit!(dep, u, g, n; factor=factor)
surface(n, title="density at t = $(t_f)")

```